EGR 426 W'26

PROJECT #3

Due Date: **At the beginning of your lecture section, April 24 at 4.30 pm**

---

1. Assignment

1.1 Objectives

- Design components of a microcontroller using behavioral descriptions

1.2 Overview

You have been given a rudimentary microcontroller architecture and implementation. You are to add to this design by implementing additional functionality. You will also develop test benches to verify the behavior of all new microcontroller instructions and functions.

You may add instructions beyond those required in your assignment if you want (in case it makes programming easier). These additional instructions do not need to have test benches.

1.3 Deliverables

- A correct, well-structured, legible, and neat top-level schematic for your design **(5%)**
- A document that clearly describes all architectural modifications you have made **(5%)**
- All VHDL files that comprise your design, properly structured, modularized and commented **(15%)**
- All VHDL test benches for your design. Your test benches must **thoroughly** test every **new** instruction/function you add and be properly structured and commented **(20%)**
- Clearly **labeled** and **annotated** timing diagrams (like the handouts/slides) showing the operation of all new instructions **(5%)**
- Compressed version of your project. Your lab instructor will compile your design, execute your design, and execute your test benches… there must be no errors!
- A live demonstration of your design, exercising all new functionality. Your demonstration must be an endless loop that constantly responds to pushbutton presses by modifying the LED displays in some fashion. This demonstration is worth **50%** of the project grade.

1.4 Suggested Approach

- Thoroughly understand the operation of the microcontroller (DO NOT SKIMP ON THIS PART!)
- Design timing diagrams and RTL code for all new instructions.
- Modify the state machine controller (if necessary) using a state transition diagram. Use StateCAD if it helps.
- Write VHDL code, test benches, etc. Do **not** start with this step first!

1.5 Extra Credit Ideas

- Use Keypad to enter data and LCD to display the result
- Write an assembler for your processor
- Write a C compiler for your processor
- Trim all unnecessary cycles from the instruction set execution models
- Eliminate the "data bus" and multiplex signals directly from source to destination. Compare resource usage (slices, RAM, etc.) with the original implementation.
- Use both the true CLK and its complement $\overline{CLK}$ to compress the cycle times such that processing takes place on rising edge of both clocks (e.g., FETCH on rising edge of CLK, EXECUTE on rising edge of $\overline{CLK}$).

---

2. New Instructions

Your assignment comprises the implementation of SOME of the following instructions (not all). The specific instructions and additional CPU functionality required will be provided separately.

2.1 ADC – Add with Carry

This instruction takes the form "ADC R" where "R" is either A or B register. This instruction adds the A and B registers together, adds in the one-bit Carry flag, and stores the result in the "R" register (A or B). You must also implement a carry bit as described in Section 3.1.

2.2 ADDD – Add Double

This instruction takes the form "ADDD M" where "M" represents a memory address. All 512 memory addresses must be allowed. This instruction adds the 16-bit number formed by concatenating the A and B registers to the 16-bit quantity formed by concatenating the contents of memory locations M and M+1. The result is stored back in the A and B registers. Condition code bits are computed accordingly.
For example, if A = 0x23 and B = 0x34 and M = 0x10E and the contents of memory addresses 0x10E and 0x10F are 0x56 and 0x78 respectively, then "ADDD 0x10E" would compute 0x2334 + 0x5678 = 0x79AC thus would store 0x79 in A and 0xAC in B.

2.3 ADDI – Add Immediate

This instruction takes the form "ADDI K, R" where "K" represents a signed, 8-bit constant and "R" is either register A or B. The instruction adds "K" to the specified register and stores the result back in the register.

2.4 ADDM – Add to Memory

This instruction takes the form "ADDM R, M" where "R" is either the A or B register and "M" is a memory location. This instruction adds the current contents of the A or B register to the contents of the memory location M and stores the result back in the memory location.

2.5 BCC – Branch on Carry Clear

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the C bit is currently clear. All 512 memory locations must be reachable by this instruction. You must also implement a carry bit as described in Section 3.1.

2.6 BCDO – BCD Output

This instruction takes the form "BCDO R" where "R" stands for either register A or B. This instruction takes the lower 4 bits of the register, decodes them as a BCD number (from 0 through 9) and sends the 7-segment LED representation of this number to Output Port #0. Similarly, the upper 4 bits of the register are decoded as a BCD number and sent as a 7-segment LED representation to Output Port #1.

2.7 BCS – Branch on Carry Set

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the C bit is currently set. All 512 memory locations must be reachable by this instruction. You must also implement a carry bit as described in Section 3.1.

2.8 BGT – Branch if Greater Than

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if register A is greater than register B, when both are interpreted as unsigned integers. All 512 memory locations must be reachable by this instruction.

2.9 BLT – Branch if Less Than

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if register A is less than register B, when both are interpreted as unsigned integers. All 512 memory locations must be reachable by this instruction.

2.10 BMI – Branch if Minus

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the N bit is set in the CCR. All 512 memory locations must be reachable with this instruction.

2.11 BNZ – Branch if Not Zero

This two-byte instruction transfers execution to a new location in memory (given in the second byte, just like for LOAD/STOR) only if the Z bit is a logic 0. All 512 memory locations must be reachable with this instruction.

2.12 BPL – Branch if Plus

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the N bit is clear in the CCR. All 512 memory locations must be reachable with this instruction.

2.13 BRR – Branch Relative

This instruction takes the form "BRR K" where "K" is a signed integer. The instruction transfers control to the memory location that is located K bytes away from the current program counter.

2.14 BRSET – Branch if Bit Set

This instruction takes the form "BRSET R, B, M". This instruction transfers execution to a new location in memory "M" only if bit number "B" (where "B" is 0 to 7) of register "R", which may be A or B, is set. All 512 memory locations must be reachable with this instruction.

2.15 BVC – Branch if Overflow Clear

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the V bit is currently clear. All 512 memory locations must be reachable by this instruction.

2.16 BVS – Branch if Overflow Set

This two-byte instruction branches to the specified memory location (given in the second byte, just like for LOAD/STOR) only if the V bit is currently set. All 512 memory locations must be reachable by this instruction.

2.17 BZ – Branch if Zero

This two-byte instruction transfers execution to a new location in memory (given in the second byte, just like for LOAD/STOR) only if the Z bit is a logic 1. All 512 memory locations must be reachable with this instruction.

2.18 CALL – Call Subroutine

This instruction requires the implementation of the hardware call stack as described in Section 3.7. The instruction takes the form "CALL M" where "M" is any 9-bit memory location. This instruction transfers control to the memory location specified by "M". Before doing so, the address of the instruction immediately following the CALL instruction is saved on the hardware call stack, pushing existing stack values down.

2.19 CC2R – Condition Codes to Register

This instruction takes the form "CC2R R" where "R" is either register A or B. This instruction transfers the condition code bits N, V, and Z (and C if the carry bit described in Section 3.1 is implemented) to the given register.

2.20 CLRB –Clear Bit

This instruction takes the form "CLRB M, B" where "M" is a desired memory location and "B" is a bit number from 0 to 7. This instruction must clear bit number "B" at memory location "M", leaving all other bits untouched. The condition code bits N and Z should be updated accordingly, while V should remain unchanged.

2.21 CLRWDT – Clear Watchdog

This instruction takes no operands. If this instruction is not executed at least every 2 seconds (or more frequently), the processor's RESET signal is asserted then negated so that the processor begins executing instructions from address 0. This instruction requires the implementation of a Watchdog peripheral as described in Section 3.5.

2.22 CMP – Compare Registers

This instruction is similar to the SUB instruction in that it performs the subtraction A – B but it does not store the result anywhere, it simply updates the condition code bits. This instruction is usually followed by a conditional branch instruction such as BNZ or BMI.

2.23 CMPI – Compare Immediate

This instruction takes the form "CMPI R, K" where "R" is either register A or B and "K" is an 8-bit constant. The instruction performs the computation R – K but does not store the result anywhere. The condition code bits are updated, however, to reflect the quantity R – K. This instruction is usually followed by a conditional branch instruction such as BNZ or BMI.

2.24 CPSE – Compare and Skip if Equal

This instruction compares the A and B registers and skips the next instruction if the two registers are equal.

2.25 DEB – Debounce

This instruction may either take the form "DEB 0, R" or "DEB 1, R" where "R" stands for either register A or B. This instruction returns the debounced status of Input #0 bit 0 (for DEB 0) or Input #0 bit 1 (for DEB 1). If this bit has remained at a logic 0 for the last 40ms, the DEB instruction should set the target register (A or B) to 1, else the target register should be set to 0.

2.26 DECMSZ – Decrement Memory and Skip if Zero

This instruction takes the form "DECMSZ M" where "M" is a memory location. This instruction decrements the contents of the memory location. If this value becomes zero, the next instruction is skipped.

2.27 DO – Begin Hardware-Assisted Loop

This instruction takes the form "DO R" where "R" is register A or B. This instruction requires the implementation of hardware-assisted loops as described in Section 3.4. The DO instruction saves the value of the given "R" register in the LC register and sets LA to be the value of the PC just after the DO instruction.

2.28 INC – Increment

This instruction takes the form "INC M" where "M" is a memory address. All 512 addresses must be allowed. This instruction increments the 8-bit number at the memory location specified and stores the result back in memory. Condition code bits are updated accordingly.

2.29 INCMSZ – Increment Memory and Skip if Zero

This instruction takes the form "INCMSZ M" where "M" is a memory location. This instruction increments the contents of the memory location. If the value becomes zero, the next instruction is skipped.

2.30 JMP – Jump

This instruction takes the form "JMP K" and transfers execution to a new memory address "K". All 512 memory locations must be reachable with this instruction.

2.31 JMPR – Jump Register

This instruction takes the form "JMPR P, R" where "R" is either register A or B and "P" is either 0 or 1. This instruction transfers control to the memory location stored in the A or B register, in the memory bank given by "P".

2.32 JMPX – Jump Indexed

This instruction requires the implementation of indexed addressing as described in Section 3.6. The instruction takes the form "JMPX R" where "R" is either register A or B. The instruction transfers control to the memory address formed by adding the X register and the "R" register.

2.33 JSR – Jump to Subroutine

This instruction transfers execution to a new location in memory. All 512 memory locations must be reachable with this instruction. The address to return to (with RTS, see Section 2.45) is stored in 2 memory locations at the address currently stored in SP. After JSR executes, the SP register should be decremented by 2. You must also implement the SP stack pointer as described in Section 3.3.

2.34 LDSP – Load Stack Pointer

This instruction requires the implementation of a software stack as described in Section 3.3. The instruction takes the form "LDSP P, K" and stores the 9-bit value formed by concatenating the 1-bit P operand and the 8-bit K operand into the SP register.

2.35 LDX – Load X Register

This instruction requires the implementation of indexed addressing as described in Section 3.6. The instruction takes the form "LDX P, K" where "P" is either 0 or 1 and "K" is a constant integer. The 9-bit value formed by "P" and "K" is stored in the X register.

2.36 LLZ – Leave Loop if Zero

This instruction requires the implementation of hardware-assisted loops as described in Section 3.4. This instruction does nothing unless the Z bit is set. If the Z bit is set, then all subsequent instructions are ignored until a LOOP instruction is encountered. The LOOP instruction is ignored, then all subsequent instructions are once again executed as normal.

2.37 LOOP – Return to Top of Loop

This instruction requires the implementation of hardware-assisted loops as described in Section 3.4. This instruction decrements the LC register and if it does not become 0, then program control is transferred to the address stored in the LA register. Otherwise, if LC becomes 0, the program continues with the next instruction following LOOP.

2.38 MAC – Multiply and Accumulate

This instruction takes the form "MAC M" where "M" represents a memory address. This instruction multiplies the current contents of the A and B registers (interpreted as unsigned numbers), adds the 16-bit result to the 16-bit contents of memory locations M and M+1, then stores the result back to these memory locations.

2.39 MOVM – Move Memory

This instruction takes the form “MOVM M1, M2”. This instruction copies the contents of the 9-bit memory address “M1” to the 9-bit memory address “M2”.

2.40 MUL – Multiply

This instruction multiplies the current contents of the A and B registers (interpreted as unsigned numbers) and stores the 16-bit result back into the A and B registers, with the A as the MSB and B as the LSB.

2.41 PWMD – PWM Duty Cycle

This instruction requires the implementation of the PWM Controller as described in Section 3.9. The instruction takes the form “PWMD R” where “R” is either register A or B. The value of the register is used to set the PWM Controller’s duty cycle. Values between 0 and 100 map directly to duty cycle percentages. Values of 101 and higher are illegal and are equivalent to a duty cycle of 0.

2.42 REST – Restore Register

This instruction takes the form “REST R” where “R” is either register A or B. This instruction pops the last value pushed using SAVE (see Section 2.46) to the register specified in the instruction. The 8-bit value popped from the SAVE stack is then removed from the stack in LIFO fashion.

2.43 RET – Return from Subroutine

This instruction requires the implementation of the hardware call stack as described in Section 3.7. The instruction takes the form “RET R, K” where “R” is either A or B register and “K” is an 8-bit constant. This instruction loads the value of “K” into A or B then transfers control to the memory location stored on the top of the hardware stack. Then, the top of the hardware stack is discarded and existing stack values moved up.

2.44 RTI – Return from Interrupt

This instruction requires the implementation of an interrupt mechanism as described in Section 3.8. The instruction restores the state of the processor and transfers control back to the instruction that was interrupted.

2.45 RTS – Return from Subroutine

This instruction transfers execution to the next instruction following the last JSR executed (see Section 2.33). The RTS instruction must automatically increment the SP register by 2 and retrieve the stored return address from memory. You must also implement the SP stack pointer as described in Section 3.3.

2.46 SAVE – Save Register

This instruction takes the form "SAVE R" where "R" is either A or B. This instruction stores the current value of the register on a 4-word LIFO (last-in-first-out) stack. That is, up to four 8-bit values may be stored on this stack. If more than 4 values are saved, the first to be saved is lost. Values from the stack are restored with the REST instruction (see Section 2.42).

2.47 SETB – Set Bit

This instruction takes the form "SETB M, B" where "M" is a desired memory location and "B" is a bit number from 0 to 7. This instruction must set bit number "B" at memory location "M", leaving all other bits untouched. The condition code bits N and Z should be updated accordingly, while V should remain unchanged.

2.48 SKIPZ – Skip if Zero

This instruction takes the form "SKIPZ" with no operands. This instruction causes the instruction immediately following to be ignored if the Z bit is currently set.

2.49 SKPMZ – Skip if Memory is Zero

This instruction takes the form "SKPMZ M". This instruction inspects the contents of memory location "M" and if the contents are 0, the next instruction is skipped.

2.50 SWAP – Swap Registers

This instruction has no operands and simply swaps the contents of registers A and B.

2.51 EBRS – Emit Burnt Resistor Smell

This instruction causes the hardware to overheat and possibly catch on fire.

2.52 BM – Branch Maybe

This instruction takes the form "BM M" and occasionally causes branches to memory address "M".

2.53 CIZ – Clear If Zero

This instruction takes the form "CIZ R" where "R" is either the A or B register. This instruction clears the specified register but only if the register has a value of 0.

---

3. CPU Functionality Enhancements

3.1 Carry Bit

The CPU is to implement a carry bit C (to add to the N, Z, and V bits) that is affected by ALU operations. Specifically, the C bit will be affected as follows:

- The ADD and SUB instructions set C if a carry (for ADD) or borrow (for SUB) would be generated as a result of the ***unsigned*** addition or subtraction of the operands. For example, 239 + 57 should set the C bit, but 127 + 10 should not. Similarly, computing 14 – 20 should set the C bit, but 13 – 3 should not.

- The LSL and LSR instructions use the C bit as the destination of the bit that was previously lost. That is, LSL shifts the MSB of the register to be shifted into the C bit, and LSR shifts the LSB of the register to be shifted into the C bit.

- The COM instruction always sets the C bit.

- The CLR instruction always clears the C bit.

- All other instructions do not modify the C bit.

3.2 Special Function Register / Timer

The Special Function Register (SFR) is to be an 8-bit register that exists at memory location 511. Rather than accessing memory at location 511, the LOAD/STOR instructions access this register. Additionally, a free-running 8-bit timer is to be implemented and exists at memory location 510. The current timer value can be both read and written at this location.

The bits of the SFR are interpreted as follows:

- **Bit 0: Timer Enable**. This bit allows the 8-bit timer peripheral to count on every clock cycle. When this bit is 0, the timer is stopped at its current count.

- **Bit 1: Timer Reset**. When this bit is set to 1 by software, the timer is reset to 0. This bit is automatically cleared and always reads 0.

- **Bit 2: Timer Overflow**. This bit is set to 1 (and remains 1 until cleared) when the timer changes from 0xFF to 0x00. This bit may be cleared by software by writing a 1 to it.

- **Bit 3 through 6: Timer Prescaler**. These bits determine how fast the built-in timer peripheral counts. A value of 0000 means the timer counts on every clock cycle. A value of N (a 4-bit unsigned number) means the timer increments once every 2^N clock cycles, so that 1111 increments the time once every 32768 clock cycles.

- **Bit 7: Sleep**. Setting this bit to 1 puts the microcontroller to sleep. The clock is disabled and no instructions execute. This bit is reset and the microcontroller is awakened when either bit #0 or bit #1 of Input #0 are asserted (i.e., the hardware pushbuttons are pressed).

3.3 Software Stack Pointer

The CPU is to implement a new 9-bit register SP. This register may be loaded with the (new) LDSP instruction described in Section 2.34. The SP register is to be used for subroutine call / return operations.

3.4 Hardware – Assisted Loops

Hardware-assisted loops are to be implemented using two additional registers (LA and LC) and new instructions. The LA register is a 9-bit register that stores the address of the beginning of the loop (i.e., the PC value at the loop start). The LC register is an 8-bit register that stores the loop count, i.e., how many more times the loop is to execute.

Neither the LA or LC registers are directly readable or writable. The "DO" instruction (see Section 2.27) sets the LA register to be the PC immediately after the "DO" instruction and the LC register to be the operand of the instruction (the number of times to iterate the loop). The "LOOP" instruction (see Section 2.37) decrements LC and if not yet zero, returns control back to the address stored in the LA. The LLZ instruction (see Section 2.36) causes all subsequent instructions up to a LOOP instruction to be ignored if the Z bit is set.

3.5 Watchdog Timer

A free-running timer is implemented that must be reset periodically using the CLRWDT instruction (see Section 2.21). If no CLRWDT instruction is executed within 2 seconds from the last CLRWDT instruction, the processor is automatically reset.

3.6 Indexed Addressing

You are to implement a new 9-bit register X. This register may be modified with the LDX instruction (see Section 2.35).

The LOAD and STOR instructions must be augmented with a new addressing mode involving the X register. The "LOAD R, X+" instruction loads the A or B register (specified by "R") with the contents of the memory location whose address is stored in the X register. After this transfer takes place, the X register is incremented by 1. Similarly, the STOR instruction must accept the addressing mode "STOR R, X+" which stores either the A or B register to the memory location whose address is stored in the X register, then increments X.

3.7 Hardware Call Stack

You are to implement an 8-level hardware call stack, not directly accessible from a user's program. This stack stores 9-bit values in a last-in-first-out (LIFO) order. The stack is manipulated by the CALL and RET instructions (see Sections 2.18 and 2.43).

3.8 Interrupts

You are to implement mechanisms to support single-level asynchronous interrupts from Input port 1. When any bit in this port changes, the current state of the processor (PC, CCR, A/B registers, etc.) must be saved on an instruction boundary and control automatically transferred to memory location 500. No more interrupts can

occur until an RTI instruction is executed (see Section 2.44). The RTI instruction is to return control back to the instruction that would have executed had the interrupt not occurred.

3.9 PWM Controller

You are to implement a hardware-assisted PWM controller that generates a 1 KHz square wave with varying duty cycle. The output of this controller is to be connected to one of the LED's on the hardware so that its brightness may be modulated. The duty cycle of the PWM controller is to be set by the PWMD instruction described in Section 2.41.